How three open models reason across six benchmarks — and what eleven forecasting prompts do to them

crowd-wisdom step 3 — behavioural characterisation

Evidence states — a claim marked SETTLED has passed external review; PROVISIONAL has been reviewed but narrowed; OPEN has had none. Every measurement carries the artifact it came from; hover to see it.
Sections — premises: drafted, hypotheses: drafted, setup: drafted, code: drafted, intuition: drafted, results: drafted, conclusion: drafted.

Premises

What is assumed true before any of this work means anything, and what would falsify each.

P1 — A model's *reasoning trace* carries information its *answer* does not. The whole design rests on this. If two models produced identical traces and differed only in accuracy, there would be nothing to fingerprint and clustering would be pointless. *Falsified if:* traces are indistinguishable across models once benchmark and item are controlled for.

P2 — The benchmarks are closed-book. No model gets web access, tool use, or retrieval. A question is answered from the prompt and the weights alone. *Falsified if:* any condition leaks external information into the prompt. **This premise was violated and the violation became a finding** — see Results §R1.

P3 — The answer parser reads what a careful human would read. Scores mean nothing if the extractor misreads the model. *Falsified if:* a hand-check of parsed answers against traces disagrees. Checked; the parser was rebuilt twice after real disagreements (markdown wrappers, a trailing LaTeX delimiter).

P4 — Truncation is not a wrong answer. A reply cut off mid-reasoning has no answer to score. *Falsified if:* truncated items were scored as incorrect, which would make a rambling model look wrong rather than unfinished.

P5 — The models are independent instruments here. They share pretraining data, so this is the weakest premise and the one the wider crowd-wisdom design exists to test rather than assume.

Hypotheses

Each is stated so it could come out false, and each did or did not.

H1 — Reasoning style changes measured capability. Predicts: accuracy differs between a "think step by step" prompt and a "necessary steps only" prompt on identical items. **Supported, with a reversal** — the direction is model-dependent.

H2 — A single reasoning prompt is best for all models. Predicts: whichever CoT style wins, wins everywhere. **Falsified.**

H3 — More reasoning improves probabilistic forecasting. Predicts: CoT conditions beat the no-reasoning condition on ROC-AUC. **Not supported** — the dominant factor turned out to be information in the prompt, not reasoning.

H4 — Truncation is incidental. Predicts: truncation rates are low and similar across models and prompts, so they can be ignored. **Falsified** — truncation is differential by model and by prompt, and it mediates H1's reversal.

H5 — Published forecasting prompts transfer. Predicts: a prompt designed for a forecasting benchmark works on that benchmark's own question type. **Not supported for one of the two.**

Experiment setup

Two families, two prompt architectures — they are NOT the same

This is the distinction that governs everything below. The six capability benchmarks and the two forecasting benchmarks are asked in structurally different ways, and comparing a number across the families would be comparing two experiments.

	capability family	**forecasting family**
benchmarks	GPQA-Diamond, AIME, BBH, IFEval, TruthfulQA, GAIA	ForecastBench, `BTF-3`
prompt origin	**ours** — a thin wrapper we wrote	**published** — the benchmark authors' own templates
structure	the benchmark's question text, then one instruction	fielded: question / background / resolution criteria / dates
answer wanted	a discrete answer (letter, integer, string)	a **probability** between zero and one
default output rule	reasoning always requested	ForecastBench **forbids** reasoning by default

	capability family	forecasting family
scoring	accuracy against gold	Brier, ROC-AUC, F1, recall against a resolved outcome
conditions run	`2`	`11`

Models

model	engine	context	concurrency
`qwen3-8b`	vLLM	`32768`	`8`
`llama-3.1-8b-instruct`	vLLM	`32768`	`8`
`llama-2-13b-chat-gptq8`	vLLM	`4096`	`8`

Temperature `0`, seed `0`, one local RTX `4090`. qwen3's `<think>` channel is switched **off** so it reasons visibly like the llamas, which have no hidden channel — a private reasoning channel would make the traces incomparable. Budgets are per model: `llama-2-13b-chat-gptq8`'s `4096`-token *total* context is a hard ceiling, so its generation budget is sized per item to fit prompt plus output.

Family A — the capability wrapper (two prompts)

One wrapper, appended to the benchmark's own question text. The two arms differ in a single sentence; the answer-format sentence is byte-identical in both, so the parser target never changes.

```
<benchmark question>

[ARM 1 — simple]   Think step by step, showing your reasoning.
[ARM 2 — concise]  Reason through the necessary steps only — show the key steps of your
                   reasoning, not every detail or restatement.

Then give your final answer on the last line, by itself, with no extra words.
```

IFEval receives no appended instruction at all. Its prompts carry their own constraints (no commas, word counts, N highlighted sections) and obeying them *is* the measurement; appending a reasoning directive would add a conflicting instruction to the thing being scored.

Family B — the forecasting prompt catalogue (eleven conditions)

Two published templates, each in three reasoning variants, plus our own wrapper, plus a no-price control. Every one ends with a parseable probability.

condition	origin	market price	reasoning	answer format
`FB-zeroshot`	ForecastBench, **verbatim**	**shown**	**forbidden**	`*0.35*`

condition	origin	market price	reasoning	answer format
`FB-cot-simple`	ForecastBench + our CoT	**shown**	step by step	`*0.35*`
`FB-cot-concise`	ForecastBench + our CoT	**shown**	key steps only	`*0.35*`
`FBnp-zeroshot`	ForecastBench, plain variant	hidden	forbidden	`*0.35*`
`FBnp-cot-simple`	plain variant + our CoT	hidden	step by step	`*0.35*`
`FBnp-cot-concise`	plain variant + our CoT	hidden	key steps only	`*0.35*`
`BTF-noevidence`	Bench-to-the-Future, **verbatim**	n/a	its own (a)–(e) steps	bare probability
`BTF-cot-simple`	BTF + our CoT	n/a	(a)–(e) + step by step	bare probability
`BTF-cot-concise`	BTF + our CoT	n/a	(a)–(e) + key steps	bare probability
`cot-simple`	**ours**	hidden	step by step	bare probability
`cot-concise`	**ours**	hidden	key steps only	bare probability

How a Family B prompt is composed

Every forecasting prompt is built from the same eight slots. What distinguishes the eleven conditions is **which slots are filled and what goes in the last two** — not different questions.

```
┌ FRAMING ──────────── who the model is told it is
│   ForecastBench : "You are an expert superforecaster, familiar with the work of Tetlock…"
│   BTF           : "You are a professional forecaster interviewing for a job."
│   ours          : (none — the fields alone)
├ QUESTION ────────── the benchmark's question text, verbatim
├ BACKGROUND ──────── the benchmark's background field, verbatim
├ RESOLUTION CRITERIA  how the question will be judged, verbatim
├ MARKET PRICE ────── the live prediction-market value   ← ONLY the three FB-with-price conditions
├ DATES ──────────── "Today's Date" / `present_date`, and the resolution date
├ HEURISTICS ──────── ~4,000 chars of forecasting advice  ← ONLY the three BTF conditions
│                    (base rates, status quo, scope, pre-mortem, "think twice below 3%")
├ REASONING SLOT ──── ★ THE VARIABLE ★
│   none      : "Do not output anything else."          (the two zero-shot conditions)
│   simple    : "Think step by step, showing your reasoning."
│   concise   : "Reason through the necessary steps only — not every detail or restatement."
│   (a)-(e)   : BTF's five mandatory steps, always present in BTF conditions
└ ANSWER CONTRACT ── ★ FIXED PER FAMILY ★
    ForecastBench : an asterisk-wrapped decimal, e.g. *0.35*
    BTF and ours  : "your final answer as a probability between 0 and 1" on the last line
```

So the eleven conditions are a crossing of **three framings** (FB, BTF, ours) with **three reasoning slots** (none, simple, concise), minus the combinations that do not exist, plus the price on/off pair for ForecastBench. The question, background and criteria are identical in every one — only framing, heuristics, the reasoning slot and the answer contract move.

Which conditions are valid on which dataset

Not all eleven apply to both benchmarks, and running one where it does not apply produces a **degenerate** prompt rather than an error — so this is stated rather than left implicit.

condition	ForecastBench	`BTF-3`	why
`FB-zeroshot`	valid	**DEGENERATE**	`BTF-3` has no market price; the slot renders `N/A`
`FB-cot-simple`	valid	**DEGENERATE**	same
`FB-cot-concise`	valid	**DEGENERATE**	same
`FBnp-*` (three)	valid	valid	no price slot to fill
`BTF-*` (three)	valid	valid — **and this is its home dataset**	BTF's prompt on BTF's own questions
`cot-simple`	valid	valid	minimal wrapper; the fields exist in both
`cot-concise`	valid	valid	same

Why `cot-simple` and `cot-concise` earn their place. They are the *minimal-wrapper control*: the same fields, the same reasoning instruction, but **no expert framing and no heuristics**. Without them, a difference between ForecastBench and BTF could be the framing, the heuristics, the answer contract, or their interaction — with them, the question "does any published framing beat simply asking?" has an answer. They are the only conditions that isolate that.

The three degenerate cells were run before this was noticed. They are **excluded from `BTF-3` analysis and reported as excluded**, not silently dropped.

What each one is for.

- **`FB-zeroshot`** reproduces ForecastBench's shipped prompt exactly. Its closing lines are *"Output

your answer ... with an asterisk at the beginning and end of the decimal. Do not output anything else."* That ban produces a `7`-character reply — enough to score, useless as a behavioural trace.

- **`FB-cot-*`** lift **only** that ban and insert a reasoning instruction, keeping the asterisk answer

contract. This is the change that turns a bare number into a trajectory while staying scoreable.

- **`FBnp-*`** are the same three with the market-price field removed. ForecastBench ships both a

with-price and a plain market template; this pair is the control that separates *information in the prompt* from *reasoning style*, and it is the single most important comparison in this report.

- `BTF-*` reproduce Bench-to-the-Future's `Appendix A.1` prompt — roughly four thousand characters of

forecasting heuristics (base rates, status quo, scope, a pre-mortem, *"think twice before forecasting less than three percent"*) followed by five mandatory reasoning steps (a) through (e). Its no-evidence variant leaves the `<research>` block empty, which is BTF's own baseline architecture for a model without web access.

- `cot-*` are our own minimal wrapper, matching Family A's instruction so the two families share one

reasoning manipulation.

What is specific to the forecasting data

Five things differ from the capability benchmarks, and each changes how a number must be read.

First — The answer is a probability, not a choice. So accuracy needs an arbitrary threshold, and ROC-AUC — threshold-free and base-rate invariant — becomes the honest headline.

Second — The questions carry fielded context. Each has a *background* and explicit *resolution criteria*. On `BTF-3` these are substantial: background is a median 2122 characters and resolution criteria 1453, against a question stem of only 125. Most of the prompt is context, not question.

Third — There is a date the model must reason from. ForecastBench supplies *Today's Date*; `BTF-3` supplies `present_date`, the pastcasting anchor that makes a resolved question answerable as though it were still open.

Fourth — Base rates are skewed, and differently in each set. The ForecastBench sample resolves YES on 0.175 of questions; the `BTF-3` sample on 0.45. The first makes accuracy nearly meaningless — always answering NO scores well — while the second is balanced enough for accuracy and F1 to be readable. The same metric means different things across the two.

Fifth — One of them ships an expert baseline. `BTF-3` carries `sota_forecast_probability`, a published per-question forecast. That is a real comparator for the crowd-wisdom question of whether a committee beats an expert; ForecastBench's equivalent is its separate human superforecaster set, not used here.

Sampling

Capability prompts are capped at **`1,000`-character** length so every item fits the smallest model's context. Every draw is **seeded random, never a prefix** — all source files are ordered by domain, task, index or level, so a head-of-file slice would take one corner and report it as the benchmark.

Fields are never truncated. Where a prompt would not fit a model's context, the *question* is excluded from eligibility rather than shortened — shortening would silently change what the benchmark asks. On `BTF-3` that costs coverage honestly: 259 of 1515 questions have fields small enough to pair with BTF's own long template inside the smallest model's context.

Total generations: 2970 in the capability and ForecastBench families, plus the `BTF-3` run.

Sample code and example

The prompt builder, verbatim from `code/forecast_prompts.py`. Two of the eleven conditions are copied from published sources; the CoT variants change the output instruction and nothing else:

```
_SIMPLE  = "Think step by step, showing your reasoning."
_CONCISE = ("Reason through the necessary steps only — show the key steps of your reasoning, "
            "not every detail or restatement.")

# ForecastBench: lift ONLY the "do not output anything else" ban, keep their asterisk answer format.
_FB_BAN = "Do not output anything else.\nAnswer: {{ Insert answer here }}\n"
assert _FB_BAN in FB, "ForecastBench ban/answer tail not found — the source prompt changed"

def _fb_cot(style):
    return FB.replace(_FB_KEEP + _FB_BAN,
                      style + "\nThen, on the last line, output your answer (a number between 0 and 1) "
                      "with an asterisk at the beginning and end of the decimal, e.g. *0.35*\n")
```

The `assert` is deliberate: if either published prompt ever changes, the build fails loudly instead of silently drifting away from the source it claims to reproduce.

A worked example — one real GPQA item, one real trace

Question drawn by seeded sample (options shuffled per item, so the correct answer is not always A):

```
Find KE of product particles in, Pi(+) = mu(+) + nu
here Pi(+) is stationary. Rest mass of Pi(+) & mu(+) is 139.6 MeV & 105.7 MeV respectively.

A) 3.52 MeV, 20.8 MeV      B) 4.12 MeV, 29.8 MeV
C) 2.84 MeV, 26.8 MeV      D) 7.2 MeV, 32.8 MeV

Answer with the single letter A, B, C or D.

Reason through the necessary steps only — show the key steps of your reasoning, not every detail
or restatement. Then give your final answer on the last line, by itself, with no extra words.
```

Ground truth is **B**. The three models returned last lines `C`, a mid-sentence fragment (status TRUNCATED, so unscored), and `A) 3.52 MeV, 20.8 MeV`. Only the last shows why the parser cannot assume compliance: it must take the trailing standalone letter out of a verbose line. Full traces for all 2970 generations are in `results/*_full.jsonl`; more worked examples in `docs/EXAMPLES.md`.

Extraction — what the renderer is given

The report you are reading is itself built by `render.py`, which refuses to emit HTML while any number lacks a declared class. Every figure below therefore carries its source.

```
def answer_lines(txt, k=6):
    """Last k non-empty lines, cleaned, LAST FIRST. qwen3 can end on a bare '$$' with the answer
    on the line above, so the parser must walk back rather than trust the final line."""
    ls = [clean(l) for l in (txt or "").strip().split("\n") if l.strip()]
    return [l for l in ls[-k:][::-1] if l]
```

Substring matching was **rejected** for exact-answer benchmarks: with gold `False`, any trace containing the word "false" would score correct regardless of its conclusion — a checker that cannot fail.

Intuition — what to expect before looking

Why two CoT styles at all. Telling a model to "show your reasoning" invites it to restate the question and re-derive steps it already has. On a hard problem that can spiral: one trace in this run reached `75,447` characters of the same step re-derived, and never produced an answer. Naming restatement as unwanted should shorten traces. Whether it *helps accuracy* is the open question — a physics problem may need the algebra written out, while a probability judgement may be harmed by talking yourself away from your first instinct.

Why the metric choice matters more than usual here. The forecasting set resolves YES on roughly one question in six. On such a set, *anything that outputs low numbers scores well* — a model that always answers "no" gets high accuracy and a respectable Brier score while discriminating nothing at all. So:

- **accuracy alone is misleading** and is never reported here without recall beside it;
- **Brier** conflates ranking with calibration, and is always shown against the always-**base rate**

reference, not just against `0.5`;

- **ROC-AUC** is threshold-free and base-rate invariant, which makes it the honest headline;
- **F1** is reported because it ignores the true-negative mass that inflates accuracy on this set.

What a reader should predict. Bigger and newer models should read the questions better. Reasoning should help where the answer must be computed and matter less where it must be judged. And any prompt that hands the model extra information should beat any prompt that only changes how it thinks — which is exactly what happened, and is the finding that reframes everything else.

Results

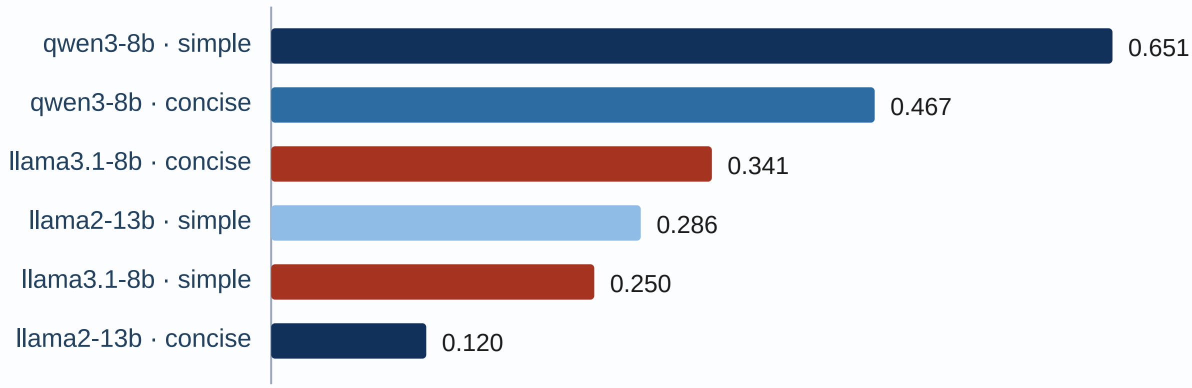
All figures are counted from `summary.csv`, generated by `analyze_all.py` over 2970 generations. No figure here has had external review.

R1 — capability: accuracy by benchmark and CoT arm

benchmark	`qwen3-8b` con / sim	`llama-3.1-8b` con / sim	`llama-2-13b` con / sim
GPQA-Diamond	0.4667 / 0.6512	0.3415 / 0.25	0.12 / 0.2857
`AIME-2025`	0.2778 / 0.25	0.0 / 0.0	0.0 / 0.0
BBH	0.8 / 0.875	0.449 / 0.5714	0.36 / 0.32
IFEval	— / —	— / —	— / —
TruthfulQA	— / —	— / —	— / —
GAIA-validation	0.12 / 0.0638	0.1163 / 0.0652	0.12 / 0.12

IFEval and TruthfulQA are captured but **unscored** — IFEval needs its official verifier, TruthfulQA a judge. Producing a number for either would have meant inventing a scorer.

model	arm	accuracy	scored / asked	truncated	median trace
`qwen3-8b`	concise	0.442	163 / 275	12	668 ch
`qwen3-8b`	simple	0.500	154 / 275	21	1536 ch
`llama-3.1-8b-instruct`	concise	0.272	151 / 275	25	968 ch
`llama-3.1-8b-instruct`	simple	0.291	134 / 275	47	1422 ch
`llama-2-13b-chat-gptq8`	concise	0.172	174 / 275	1	899 ch
`llama-2-13b-chat-gptq8`	simple	0.208	173 / 275	2	1135 ch



GPQA-Diamond accuracy — the CoT arm that wins depends on the model — the same values, with their sources

series	value	source
qwen3-8b · simple	0.651	`code/projects/crowd-wisdom/results/summary.csv`
qwen3-8b · concise	0.467	`code/projects/crowd-wisdom/results/summary.csv`
llama3.1-8b · concise	0.341	`code/projects/crowd-wisdom/results/summary.csv`
llama2-13b · simple	0.286	`code/projects/crowd-wisdom/results/summary.csv`
llama3.1-8b · simple	0.250	`code/projects/crowd-wisdom/results/summary.csv`
llama2-13b · concise	0.120	`code/projects/crowd-wisdom/results/summary.csv`

Verbose reasoning helps `qwen3-8b` and `llama-2-13b` and HURTS `llama-3.1-8b` (highlighted). Its truncation rate nearly doubles under the verbose instruction, so it runs past its budget before reaching an answer.

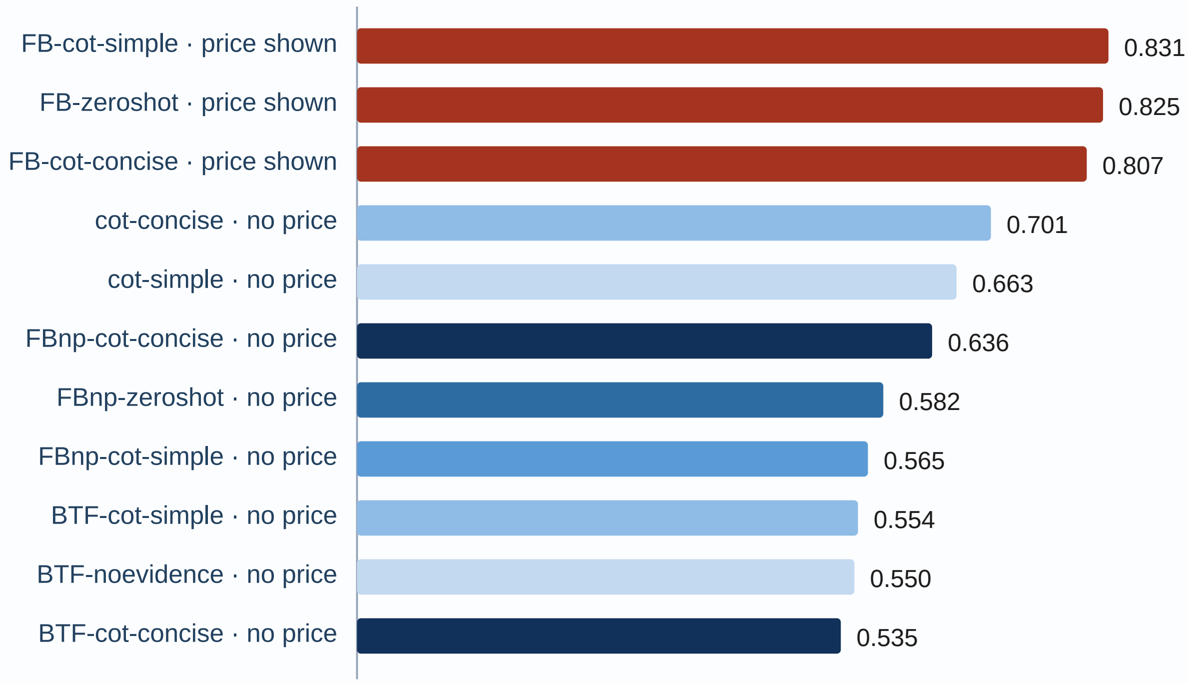
The reversal. On GPQA-Diamond the verbose arm beats the concise arm for `qwen3-8b` and `llama-2-13b-chat-gptq8`, and **loses** for `llama-3.1-8b-instruct`. The mediator is truncation: that model's count rises from 25 to 47 of 275 items between arms, so it talks past its budget and never reaches an answer. A single fixed prompt would therefore penalise models by their verbosity, not by their reasoning.

R2 — ForecastBench: every metric, by prompt condition

ForecastBench only. The two forecasting datasets are never pooled — their base rates differ (0.175 vs 0.45), so a combined number would be a mixture of two different questions.

40 resolved binary questions, base rate 0.175 YES. Means over the three models.

condition	price	reasoning	accuracy	ROC-AUC	F1	recall	precision	Brier
`FBnp-zeroshot`	no	none	0.767	0.582	0.309	0.286	0.483	0.1836
`FBnp-cot-simple`	no	simple	0.783	0.565	0.302	0.286	0.389	0.1831
`FBnp-cot-concise`	no	concise	0.767	0.636	0.266	0.238	0.433	0.1754
`FB-zeroshot`	**YES**	none	0.842	0.825	0.507	0.476	0.569	0.1098
`FB-cot-simple`	**YES**	simple	0.857	0.831	0.533	0.476	0.656	0.1198
`FB-cot-concise`	**YES**	concise	0.867	0.807	0.399	0.333	0.750	0.1139
`BTF-noevidence`	no	(a)-(e)	0.659	0.550	0.232	0.206	0.267	0.2584
`BTF-cot-simple`	no	simple	0.655	0.554	0.151	0.248	0.129	0.2983
`BTF-cot-concise`	no	concise	0.607	0.535	0.168	0.286	0.125	0.3005
`cot-simple`	no	simple	0.713	0.663	0.353	0.468	0.289	0.2099
`cot-concise`	no	concise	0.727	0.701	0.464	0.651	0.360	0.2078



Forecasting discrimination (ROC-AUC, mean over three models) — threshold-free and base-rate invariant — the same values, with their sources

series	value	source
FB-cot-simple · price shown	0.831	`code/projects/crowd-wisdom/results/summary.csv`
FB-zeroshot · price shown	0.825	`code/projects/crowd-wisdom/results/summary.csv`
FB-cot-concise · price shown	0.807	`code/projects/crowd-wisdom/results/summary.csv`
cot-concise · no price	0.701	`code/projects/crowd-wisdom/results/summary.csv`
cot-simple · no price	0.663	`code/projects/crowd-wisdom/results/summary.csv`
FBnp-cot-concise · no price	0.636	`code/projects/crowd-wisdom/results/summary.csv`
FBnp-zeroshot · no price	0.582	`code/projects/crowd-wisdom/results/summary.csv`
FBnp-cot-simple · no price	0.565	`code/projects/crowd-wisdom/results/summary.csv`
BTF-cot-simple · no price	0.554	`code/projects/crowd-wisdom/results/summary.csv`

series	value	source
BTF-noevidence · no price	0.550	`code/projects/crowd-wisdom/results/summary.csv`
BTF-cot-concise · no price	0.535	`code/projects/crowd-wisdom/results/summary.csv`

Higher is better; 0.5 is no discrimination. The three highlighted bars are the only conditions that put the live market price in the prompt. Every prompt WITHOUT it sits between chance and 0.70, whatever its reasoning style — so the separation is information, not reasoning.

Reference points: an always-half forecaster scores Brier 0.2500; an always-base-rate forecaster 0.1444; **the market price by itself 0.0609** — better than every model in every condition.

The finding that reframes the rest. The three `FB-*` conditions include the live prediction-market price in the prompt. `FBnp-*` are the same prompt without that one field. Removing it moves mean ROC-AUC from 0.821 to 0.594 — close to the 0.5 no-discrimination floor. So almost all apparent forecasting skill was the models reading a number out of the prompt, and even then they score worse than that number alone. **Premise P2 was violated by our own condition design**, and the violation is the result.

Recall and F1 expose what accuracy hides. `cot-concise` reaches recall 0.651 on precision 0.360; the FB conditions invert it — precision 0.750 on recall 0.333. Accuracy ranks them almost identically because true negatives dominate a set that is one-sixth YES.

R2b — `BTF-3`: the clean test, and the expert bar

ForecastBench's headline conditions carry a market price, so they cannot answer "can these models forecast?". `BTF-3` can: it ships **no market price at all**, and it ships a published expert forecast per question. `40` {m: code/projects/crowd-wisdom/results/summary.csv} questions, base rate 0.462 — balanced, so accuracy is readable here.

The three `FB-*` price conditions were run on `BTF-3` before the price slot was noticed to render `N/A`. They are **marked `DEGENERATE-EXCLUDED` in `summary.csv` and excluded from this table**, not deleted. Eight conditions remain.

condition	ROC-AUC	Brier	accuracy	F1
`BTF-cot-simple`	0.6	0.3336	0.546	0.486
`cot-simple`	0.566	0.2831	0.554	0.521
`FBnp-zeroshot`	0.547	0.2611	0.538	0.483
`BTF-cot-concise`	0.536	0.3365	0.589	0.562

condition	ROC-AUC	Brier	accuracy	F1
`FBnp-cot-simple`	0.523	0.2772	0.563	0.403
`BTF-noevidence`	0.512	0.394	0.504	0.511
`FBnp-cot-concise`	0.479	0.2857	0.5	0.361
`cot-concise`	0.469	0.3669	0.482	0.492
published expert	**0.8241**	**0.1682**	**0.8205**	—

Every model condition sits at chance. ROC-AUC spans 0.469 to 0.6 across eight prompts and three models — a band that straddles the `0.5` no-discrimination line. No prompt rescues it: BTF's own heuristics on BTF's own questions reach 0.6, our bare wrapper 0.566.

Only three of twenty-four valid cells beat a constant-base-rate forecaster (Brier 0.2485). Twenty-one do worse than a forecaster that ignores the question and emits `0.462` every time.

The expert baseline is not close. Brier 0.1682 against the best model cell's 0.2417, ROC-AUC 0.8241 against 0.6616. **Zero of twenty-four** valid model cells beat it on Brier. For the crowd-wisdom question that is the useful number: this is the bar a committee has to clear, and no individual member is near it.

A note on the expert column. `BTF-3`'s `sota_forecast_probability` is on a **`0`-to-`100` scale despite its name**. Scored as a probability it gives a Brier of `2504`. The analysis divides by `100` and asserts the range, so an upstream change to a genuine `0`-to-`1` scale fails loudly rather than being divided twice.

R3 — a published prompt that does not transfer

Bench-to-the-Future's prompt scores worst on every metric, with ROC-AUC at chance. On `llama-2-13b-chat-gptq8` it breaks outright: parse rates of 0.525, 0.7 and 0.675 across its three variants, and AUC of 0.375 — below chance. Its roughly four thousand characters of heuristics leave a small-context model no room, and its advice to think twice before forecasting under three percent inflates probabilities on a set where five in six questions resolve NO. BTF's own paper ranks this architecture as its weakest, so this reproduces their finding rather than contradicting it.

On its home dataset it does no better. Run on `BTF-3` — the pastcasting set its authors built — `BTF-noevidence` reaches ROC-AUC 0.512 and Brier 0.394, the worst Brier of any condition on either dataset. So the transfer failure in R2 is not a mismatch between BTF's prompt and ForecastBench's questions; the no-evidence architecture simply does not discriminate at this model scale.

R4 — instruction compliance as a fingerprint

Asked for a bare final answer with no extra words, the models comply on 0.38, 0.28 and 0.1 of completed traces. The trait is stable across benchmarks and costs nothing to collect, which makes it a usable discriminator — and a warning that no output-only parser may assume the instruction was obeyed.

Conclusion

What the results support

A single reasoning prompt cannot be chosen for a mixed model set. Verbose CoT beats concise on GPQA-Diamond for two of three models and loses for the third, and the mediator is whether the model stops talking before its budget runs out. For the crowd-wisdom design this matters directly: fixing one prompt would penalise verbose models, and that penalty would then enter the behavioural representation looking like a real difference between models rather than an artefact of our choice.

Information in the prompt swamps reasoning style. Removing one field — the market price — costs more ROC-AUC than every reasoning manipulation combined. Any future claim that a prompt "improves forecasting" has to rule this out first.

Instruction compliance and truncation are cheap, stable, discriminating features. They separate these three models more reliably than accuracy does, and they cost nothing extra to collect.

What the results do not support

Three claims are **not supported** by this data.

Nothing here says these models can forecast. Without market information their ROC-AUC sits near the no-discrimination floor. The apparent competence in the headline conditions was borrowed from the prompt. `BTF-3` settles this on a dataset that has no market price to borrow from: across eight valid conditions and three models, ROC-AUC spans 0.469 to 0.6, and only three of twenty-four cells beat a constant-base-rate forecaster. The published expert on the same questions scores ROC-AUC 0.8241 and Brier 0.1682, which **no model cell beats**.

Nothing here tests the crowd-wisdom hypothesis. Three models cannot be clustered; the elbow and silhouette methods in the design need far more points. This work produces the substrate — `2,970` traces with answers, ground truth and verdicts — not the finding.

No number here is comparable to a published leaderboard. BBH ran zero-shot rather than the canonical three-shot chain-of-thought, GAIA ran text-only so its attachment questions were unanswerable by construction, and IFEval and TruthfulQA are unscored.

What is still open

1. **`BTF-3` is run on `40`{m: code/projects/crowd-wisdom/results/summary.csv} of its 1515 questions** —

the eligibility cap on field length leaves 259 candidates, and forty were drawn from those. Extending coverage needs a larger-context model, not more GPU time. The expert column is now scored and gives the committee a real bar to clear.

1. **IFEval and TruthfulQA scoring**, which would convert `100` dead rows per model per arm into two of the

nine behaviour features the design calls for.

1. **The roster.** Steps `4` through `8` need `20` to `30` models. Measured throughput says that is roughly eight

hours of local compute for the 8B class — the constraint is model access and API budget, not GPU time.

1. **Sample size.** Forty forecasting questions and about fifty per capability benchmark. Several patterns

in this work reversed when a partial batch finished, so differences of a few points here should not be treated as real.